Laborator 07 - CI/CD & Documentation

Documentatia

Comentariile

Comentariile sunt printre cele mai folosite si abuzate moduri de a documenta codul. Desi acestea nu sunt recomandate pentru toate cazurile [https://www.youtube.com/watch?v=Bf7vDBBOBUA] acestea pot fi folosite pentru a ascunde diverse imperfectiuni ale codul sau proiectului ca nume neinspirate ale variabilelor sau functiilor, design slab al codului sau lipsa uneltelor de proces (sistem de versionare, issue tracking etc.). Comentariile pot fi utile pentru a explica de ce este facut un lucru in cod, pentru a documenta un design bazat pe pseudo-cod sau pentru a referentia lucruri conexe.

Tipuri de comentarii

- Repetare a codului
  - Inutile
- Explicarea codului
  - Utile in cazul in care codul nu este clar, dar in acest caz prioritatea ar fi clarificarea codului
- Markers
  - Note menite sa fie scoase din codul final
  - Utile daca sunt standardizate ex. `// TODO`
- Rezumat al codului
  - Aplicate pentru sectiuni mai mari de cod
  - Utile pentru a intelege mai repede si usor codul
- Descriere a unei potentiale probleme
  - Asemanator cu rezumatul, doar ca se descrie o potentiala problema
- Alte informatii care nu pot fi exprimate in cod
  - Numele autorilor, copyright, data modificarii etc.

Documentarea unui API

Uneltele de documentare a API-urilor au devenit extrem de comune in prezent. Acestea:

- Reflecta accentul modern pus pe interfetele reutilizabile
- Combina informatii de la
  - Un parser limitat de limbaj (pentru a extrage informatii despre structura modulului/functiei si parametrii asociati
  - Sectiuni de comentarii special formatate din codul sursa
- Incurajeaza actualizarea comentariilor pe masura ce codul este modificat

DoxyGen

Unul dintre cele mai cunoscute unelte de documentare a API-urilor. In acest laborator vom folosi Docstring-uri [https://peps.python.org/pep-0257/] fiind cea mai utilizata conventie de scriere a comentariilor in cadrul limbajului Python. DoxyGen va genera documentatia automat folosindu-se de prezenta comentariilor in format Docstring [https://peps.python.org/pep-0257/].

- Sunt delimitate de `"""..."""`, implicit procesate ca si comentarii
- Preced sectiunea de cod pe care o comenteaza
- Pe langa orice text, acestea pot contine marcaje speciale cum ar fi `@author <authorName>`, `@param <name> <description>`, `@return <description>` etc.

Pentru a instala DoxyGen puteti folosi comanda:

```
sudo apt install doxygen
```

Pentru a putea genera documentatia aceasta are nevoie de un fisier de configurare, Doxyfile. Pentru a genera un template pentru acest fisier puteti folosi comanda:

```
doxygen -g
```

Pentru a rula DoxyGen, puteti rula direct in terminal:

```
doxygen
```

Daca sunt necesare alte configuratii, atunci va fi necesara schimbarea fisierului Doxyfile.

Continuous Integration

In cadrul Continuous integration practicile de version control, compilarea si testarea automata sunt combinate astfel incat schimbarile care ajung in repository-ul de version control sunt automat verificate, testate si regenerate rapoartele aferente. Este de preferat sa se faca aceste lucruri complet sau partial pentru a reduce riscul ca schimbarile noi sa introduca probleme ca erori de compilare sau bug-uri. Pentru ca sunt multi pasi necesari in a executa un plan de Continuous Integration, durata poate sa fie extrem de mare. Datorita acestui fapt, de fiecare data cand se introduc schimbari noi se va prefera rularea unui plan redus(de exemplu care include compilarea si analiza statica, dar nu include testare si analiza dinamica). Planul complet poate sa fie rulat ocazional, cum ar fi peste noapte, cand poate sa dureze mult mai mult. Pentru a putea beneficia de Continuous Integration, un proiect trebuie sa aiba mai multe caracteristici:

- Sistem de control al versiuni cu branch principal clar identificat sau set de branch-uri de development
- Build automat
- Dezvoltatori care introduc schimbari frecvent (poate de mai multe ori pe zi)
    - Commit-uri introduse pe branch-uri urmarite (ex. cu pull-request asociat) sunt verificate cu un plan minimal de Continuous Integration (compiling, static analysis)
- Testarea se face, ideal, pe o clona a mediului de productie
    - Diferit de mediul de dezvoltare
    - De obicei verificat rar
    - Poate sa foloseasca mai multe masini de rulare pentru a putea viza sisteme cu configuratii diferite sau pentru a paraleliza volumul de munca

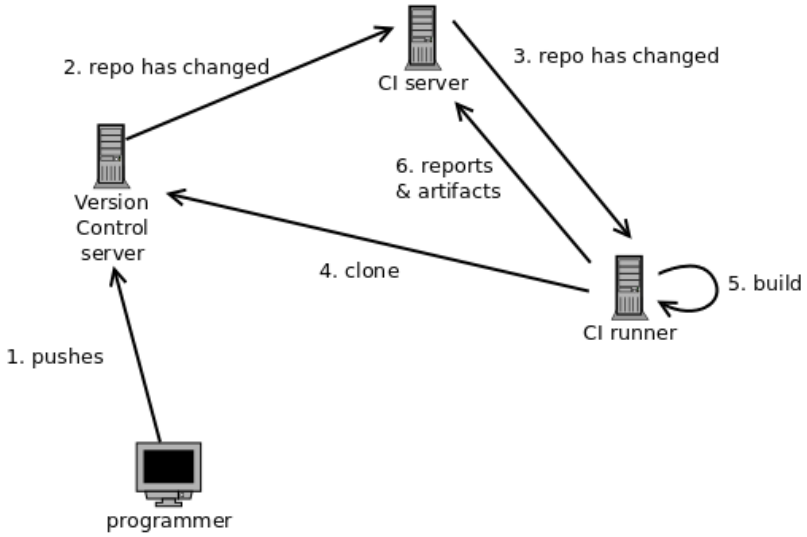
Avantaje:

- Problemele de integrare sunt identificate si rezolvate repede
- Schimbarile sunt testate cat mai repede posibil
- Se pune accentul pe check-in-uri frecvente, astfel se incurajeaza modularitatea

Dezavantaje:

- Efort initial ridicat pentru a face setup la sistem
- In functie de structura proiectului, e necesar un anumit nivel de rafinament pentru a pune fiecare pas de verificare intr-un build automat

Sistem de Continuous Integration



Un server de Continuous Integration este o masina accesibila din retea care:

- Are accesul la informatiile legate de proiectele care trebui rulate, locatia si informatiile de acces la repository-ul de version control, ce branch-uri sa urmareasca, cum sa faca build la proiect si ce rapoarte trebuie sa produca
- Monitorizeaza repository-ul de version control pentru a vedea noi commit-uri
- Cand un push la un branch urmarit apare, server-ul de CI notifica un runner

Un CI runner (sau nod) e un proces pe o masina care:

- Are uneltele necesare pentru a face build la proiect
- Este gestionat de serverul de CI

Cand un runner este notificat de serverul de CI, acesta:

- Cloneaza branch-ul pe care se doreste rularea build-ului din repository-ul de version control
- Ruleaza build-ul
- Publica rapoartele rezultate build-ului
    - Poate publica si “artefacte”: fisiere generate care contin informatii aditionale cum ar fi versiunea uneltelor folosite in cadrul build-ului

GitHub Actions

GitHub furnizeaza o solutie integrata de Continuous Integration printr-un serviciu numit “Actions”.
Acest serviciu poate fi accesat din pagina proiectului, selectand tab-ul Actions, de exemplu [https://github.com/sjzeil/CoWeM/actions].
Pentru a creea un Action nou, este necesara creearea unui fisier de workflow la calea `.github/workflows/` din repository. Acest lucru se poate face atat local cat si din GitHub. Fisierul de workflow are extensia `.yml` pentru ca foloseste sintaxa YAML [https://en.wikipedia.org/wiki/YAML] si contine indicatii legate de cand si ce este executat in cadrul unui action. Un exemplu de fisier de workflow gasiti aici [https://github.com/Ingineria-Calculatoarelor-ACS-UPB/tree-traversal/blob/main/.github/workflows/run.yml].
Fisierele ce pot rezulta in urma rularii anumitor pasi pot fi publicate ca artefacte si in cadrul GitHub Actions. Pentru a face acest lucru, puteti folosi urmatoarea sintaxa, dupa pasul de generare al fisierului dorit:

```
- name: Upload a Build Artifact
    uses: actions/upload-artifact@v3.0.0
    with:
      # Artifact name
      name: # optional
      # Destination path
      path: # optional
```

Generarea unui artefact de build este o extensie a sistemului oferit de GitHub. Aici [https://github.com/marketplace?type=actions] gasiti marketplace-ul cu toate extensiile disponibile pentru GitHub Actions.

Exercitii

1. Clonati repo-ul [https://github.com/Ingineria-Calculatoarelor-ACS-UPB/tree-traversal].
2. Folositi DoxyGen pentru a genera documentatia deja existenta in acest proiect. Verificati fisierul `html/index.html` pentru a observa ce functii sunt deja documentate.
3. Adaugati Docstrings si pentru celelalte functii din fisierul `tree.py`.(TODO 1)
4. Implementati functiile `_printPreorderTree` si `_printPostorderTree` si scrieti cel putin 2 unit teste pentru functia `_find`.
5. In GitHub, creati un action care ruleaza unit testele pentru fiecare commit nou adaugat pe branch-ul `main`. Pentru a putea face acest lucru, faceti un fork al repository-ului si actualizati branch-ul `main` al fork-ului nou creat.
6. Actualizati action-ul creat anterior, astfel incat acesta sa se ruleze numai atunci cand se creaza un pull request.(Hint [https://docs.github.com/en/actions/using-workflows/events-that-trigger-workflows#pull_request])
7. Creati o noua actiune care ruleaza doxygen, doar atunci cand se creeaza un nou tag cu o noua versiunea a aplicatiei, si salveaza documentatia ca un artefact.(Hint [https://github.com/marketplace/actions/upload-a-build-artifact])

Este recomandat sa folositi un fisier .gitignore care va ignora fisierele ce nu sunt necesare a fi incarcate in repo (cum ar fi fisierele executabile compilate). Acest fisier trebuie adaugat in folder-ul repository-ului (preferabil in root-ul acestuia) si adaugat in remote cu urmatorul commit. Un exemplu de continut pentru acest fisier gasiti aici [https://pastebin.com/hdiHVEMS].

tsc/laboratoare/laborator-07.txt · Last modified: 2024/04/24 16:23 by giorgiana.vlasceanu